

QUAMC

Quality Assurance Management Center

System Design Document

University Accreditation Document Management System

Stack: React · TypeScript · Inertia.js · Laravel 11 · Tailwind CSS · MySQL

React + TS

Inertia.js

Laravel 11

MySQL

Redis

Tailwind

Table of Contents

1	System Overview	3
2	Core Roles & RBAC	3
3	Accreditation Areas & Programs	4
4	System Architecture	4
5	Document Workflow	5
6	Database Schema	6
7	API Structure	7
8	Folder Structure	8
9	Dashboard Features	9
10	Example Workflow Scenario	9
11	Advanced Features	10
12	Technology Stack Summary	10

1 System Overview

QUAMC (Quality Assurance Management Center) is a web-based accreditation document management system for universities. It manages document submissions, multi-stage approval workflows, and real-time accreditation readiness dashboards across multiple academic programs and configurable accreditation areas.

Objective	Detail
Multi-program support	BSIT, BSBA, Nursing, Education and any future program
Dynamic area management	Accreditation areas are configurable, not hardcoded
Hierarchical approval	Area Coord → Program Coord → Dean → Director
File versioning	Every document revision is stored with full history
Audit trail	Every action logged with timestamp, user, and IP address
Role-based access	Five distinct roles with Laravel Policies and Gates

2 Core Roles & RBAC

Five roles are enforced via Laravel Policies and Gates. Each role has a strictly scoped set of permissions — no role can perform actions outside its defined boundaries.

System Admin

- Create and manage user accounts
- Assign roles to users
- Assign coordinators to any area globally
- Manage accreditation areas and system config

QUAMC Director

- Oversees the entire system globally
- View all programs and areas
- Monitor accreditation progress
- Final approval or return of documents

Dean / Office Head

- Manage quality assurance for their college
- Configure areas, sub-areas, and items within their college
- Assign Area Coordinators to areas within their college
- Review submissions from Program Coordinators
- Forward approved documents to Director

Program Area Coordinator

- Monitor multiple areas within a program (e.g. BSIT)
- Review documents from Area Coordinators
- Provide feedback and forward to Dean

Area Coordinator

- Responsible for one specific accreditation area
- Upload and manage evidence files
- Submit documents for review

Permission matrix

Action	Area Coord	Program Coord	Dean	Director	Admin
Upload file	Yes	-	-	-	-
Submit document	Yes	-	-	-	-
Review & forward	-	Yes	Yes	-	-
Return for revision	-	Yes	Yes	Yes	-

Final approve	-	-	-	Yes	-
Archive document	-	-	-	Yes	-
Assign area coordinator	-	-	College only	-	Yes (global)
Configure areas	-	-	College only	-	Yes (global)
Configure sub-areas	-	-	College only	-	Yes (global)
Configure items	-	-	College only	-	Yes (global)
Manage users	-	-	-	-	Yes
View all programs	-	-	College	Yes	Yes

College-scoped enforcement: All Dean actions on areas, sub-areas, items, and coordinator assignments are validated server-side via Laravel Policies. The AreaPolicy checks that the target area belongs to a program under the Dean's college before allowing create, update, or delete. A Dean cannot configure or assign within another college even if they know the IDs.

3 Accreditation Areas & Programs

Areas are fully dynamic — they live in the database and are assigned to programs by the System Admin. No area is hardcoded in the codebase. Each area follows a strict 3-level hierarchy: Area contains Sub-areas, Sub-areas contain Items, and Items are the only level where evidence documents are uploaded.

Document hierarchy

Level	Table	Example	Holds files?
1 — Area	areas	Area 3 – Curriculum	No — container only
2 — Sub-area	sub_areas	3.1 Course Offerings	No — container only
3 — Item	area_items	CMO alignment matrix	YES — files attach here
4 — Document	documents	cmo_matrix_v2.pdf	Via document_versions

#	Area Name	#	Area Name
1	Vision, Mission, Goals	6	Extension
2	Faculty	7	Library
3	Curriculum	8	Physical Facilities
4	Students	9	Administration
5	Research	10	Quality Assurance

Supported programs

Program	Code	Areas	Status
Bachelor of Science in Information Technology	BSIT	10	Active
Bachelor of Science in Business Administration	BSBA	10	Active
Bachelor of Science in Nursing	BSN	10	Active
Bachelor of Secondary Education	BSED	10	Active

Concrete example — BSIT Area 3

Area	Sub-area	Item (files attach here)
Area 3 Curriculum	3.1 Course Offerings	List of all courses offered CMO alignment matrix Course description per subject
	3.2 Syllabus Quality	Sample syllabi (5 courses) Faculty endorsement of syllabi
	3.3 CMO Compliance	CMO No. and date of effectivity Compliance matrix per CMO

4 System Architecture

Client Layer	React 18 + TypeScript · Inertia.js SPA · Tailwind CSS · ShadCN/UI · Ziggy + Vite
Inertia.js Bridge	Server-side rendering handoff · Shared props · Form helpers · No separate REST API for frontend
Laravel Application Layer	Controllers (Inertia responses) · Services (business logic) · Policies + Gates (RBAC) · Jobs + Events (queued notifications) · Observers (activity log) · Middleware stack
Data Layer	MySQL (primary data store) · Redis (cache, queues, sessions) · Local Disk dev / Cloudflare R2 production (evidence files) · Meilisearch optional (full-text search)
Infrastructure	Nginx / Apache · PHP 8.3 + OPcache · Queue Worker · Laravel Horizon + Scheduler

Middleware stack

- Auth — Sanctum session / token authentication
- CheckRole — validates role before controller executes
- Throttle — rate limiting on API routes
- InertiaShare — passes auth user, notifications, flash messages to every page
- LogActivity — captures request metadata into activity_logs table

5 Document Workflow

Every document follows a strict hierarchical approval chain. Each stage can approve, return with comments, or forward to the next level. The WorkflowService owns the state machine and validates all transitions.



Return arrows: any reviewer at any stage can return the document to the Area Coordinator with comments. A new version resets the workflow.

Document status states

Status	Description
Draft	Document created but not yet submitted. Editable by Area Coordinator.
Pending Review	Submitted and awaiting action at the current workflow stage.
Approved	Final approval granted by QUAMC Director.
Returned	Sent back to Area Coordinator with reviewer comments. Must be revised.
Archived	Approved and archived for historical record. Read-only.

6 Database Schema

users	column	type	notes
	id	<i>uuid</i>	PK
	name	<i>string</i>	
	email	<i>string</i>	unique
	password	<i>string</i>	hashed
	is_active	<i>boolean</i>	default true
	created_at / updated_at	<i>timestamp</i>	

roles	column	type	notes
	id	<i>int</i>	PK auto-increment
	name	<i>string</i>	e.g. Area Coordinator
	slug	<i>string</i>	e.g. area-coordinator

programs	column	type	notes
	id	<i>int</i>	PK
	name	<i>string</i>	
	code	<i>string</i>	e.g. BSIT
	is_active	<i>boolean</i>	

areas	column	type	notes
	id	<i>int</i>	PK
	program_id	<i>int</i>	FK -> programs
	name	<i>string</i>	e.g. Area 3 - Curriculum
	order_number	<i>tinyint</i>	
	is_active	<i>boolean</i>	

sub_areas	column	type	notes
	id	<i>int</i>	PK
	area_id	<i>int</i>	FK -> areas
	name	<i>string</i>	e.g. 3.1 Course Offerings
	order_number	<i>tinyint</i>	
	is_active	<i>boolean</i>	

area_items	column	type	notes
	id	int	PK
	sub_area_id	int	FK -> sub_areas
	name	string	e.g. CMO alignment matrix
	description	text	guidance text for coordinator
	order_number	tinyint	
	is_required	boolean	required vs optional evidence
	is_active	boolean	

area_assignments	column	type	notes
	id	int	PK
	user_id	uuid	FK -> users — the coordinator being assigned
	area_id	int	FK -> areas — assigned at area level
	assigned_by	uuid	FK -> users — Admin or Dean who assigned
	role_type	string	area_coord program_coord
	academic_year	int	

documents	column	type	notes
	id	uuid	PK
	area_item_id	int	FK -> area_items - leaf level only
	uploaded_by	uuid	FK -> users
	title	string	
	status	string	draft pending_review approved returned archived
	current_version	int	reflects latest version_number
	submitted_at	timestamp	nullable

document_versions	column	type	notes
	id	uuid	PK
	document_id	uuid	FK -> documents
	uploaded_by	uuid	FK -> users
	version_number	int	scoped per document - never overwritten
	file_path	string	dev: storage/app/ prod: Cloudflare R2 path
	original_filename	string	
	file_size_bytes	bigint	
	mime_type	string	
	notes	text	nullable

workflow_actions	column	type	notes
	id	<i>uuid</i>	PK
	document_id	<i>uuid</i>	FK → documents
	actor_id	<i>uuid</i>	FK → users
	action	<i>string</i>	submit approve return forward archive
	from_status	<i>string</i>	
	to_status	<i>string</i>	
	comment	<i>text</i>	nullable
	acted_at	<i>timestamp</i>	

notifications	column	type	notes
	id	<i>uuid</i>	PK
	user_id	<i>uuid</i>	FK → users
	document_id	<i>uuid</i>	FK → documents nullable
	type	<i>string</i>	
	message	<i>text</i>	
	is_read	<i>boolean</i>	default false
	created_at	<i>timestamp</i>	

activity_logs	column	type	notes
	id	<i>uuid</i>	PK
	user_id	<i>uuid</i>	FK → users
	event	<i>string</i>	e.g. document.submitted
	model_type	<i>string</i>	
	model_id	<i>uuid</i>	
	changes	<i>json</i>	before/after values
	ip_address	<i>string</i>	
	created_at	<i>timestamp</i>	

7 API Structure

Inertia.js routes handle the web SPA. Laravel Sanctum token routes provide REST API access for any future mobile or third-party integrations.

Documents		
GET	/documents	List documents (filtered by role scope)
GET	/documents/{id}	Show document with version history
POST	/documents	Create new document
POST	/documents/{id}/versions	Upload new file version
POST	/documents/{id}/submit	Submit document for review
POST	/documents/{id}/workflow	Perform workflow action (approve/return/forward)
GET	/documents/{id}/versions/{vid}/download	Secure file download
GET	/documents/{id}/history	Full workflow action history

Programs & Areas		
GET	/programs	List all programs
POST	/programs	Create program (Admin)
GET	/programs/{id}/areas	List areas for a program
POST	/areas	Create area (Admin)
POST	/areas/{id}/assign	Assign coordinator to area
DELETE	/areas/{id}/assign/{uid}	Remove assignment
GET	/areas/{id}/progress	Area accreditation progress

Users & Admin		
GET	/users	List users (Admin)
POST	/users	Create user (Admin)
PUT	/users/{id}	Update user
POST	/users/{id}/roles	Assign role to user
GET	/notifications	Current user notifications
POST	/notifications/{id}/read	Mark notification as read
GET	/activity-logs	System activity log (Admin/Director)

Dashboards & Reports		
GET	/dashboard/director	Global accreditation overview
GET	/dashboard/dean	Department progress
GET	/dashboard/coordinator	Area status for current user
GET	/reports/readiness	Accreditation readiness report
GET	/reports/readiness/export	Export PDF report

8 Folder Structure

Laravel Backend

```
app/
  ■■■ Http/
  ■ ■■■ Controllers/ Inertia + API controllers
  ■ ■■■ Middleware/ Auth, CheckRole, LogActivity
  ■ ■■■ Requests/ Form request validation
  ■■■ Models/ Eloquent models
  ■■■ Services/ Business logic layer
  ■ ■■■ DocumentService.php
  ■ ■■■ WorkflowService.php
  ■ ■■■ FileVersionService.php
  ■ ■■■ NotificationService.php
  ■■■ Policies/ RBAC authorization
  ■■■ Events/ & Listeners/ Async event handling
  ■■■ Observers/ Model event hooks

database/
  ■■■ migrations/ All schema migrations
  ■■■ seeders/ Dev seed data

storage/app/documents/ DEV: evidence files
  ■■■ {program}/{area}/{item}/v{n}_{hash}.ext
  (PROD: swap to Cloudflare R2 via .env)

routes/
  ■■■ web.php Inertia routes
  ■■■ api.php Sanctum REST routes
```

React Frontend

```
resources/js/
  ■■■ Pages/
  ■ ■■■ Auth/Login.tsx
  ■ ■■■ Dashboard/
  ■ ■ ■■■ Director.tsx
  ■ ■ ■■■ Dean.tsx
  ■ ■ ■■■ Coordinator.tsx
  ■ ■■■ Documents/
  ■ ■ ■■■ Index.tsx
  ■ ■ ■■■ Show.tsx
  ■ ■ ■■■ Upload.tsx
  ■ ■ ■■■ Versions.tsx
  ■ ■■■ Areas/
  ■ ■■■ Programs/
  ■ ■■■ Users/
  ■■■ Components/
  ■ ■■■ ui/ ShadCN components
  ■ ■■■ DocumentCard.tsx
  ■ ■■■ WorkflowTimeline.tsx
  ■ ■■■ ProgressBar.tsx
  ■ ■■■ FileUploader.tsx
  ■ ■■■ NotificationBell.tsx
  ■■■ Layouts/
  ■ ■■■ AppLayout.tsx
  ■ ■■■ AuthLayout.tsx
  ■■■ hooks/
  ■ ■■■ useRole.ts
  ■ ■■■ useDocumentStatus.ts
  ■■■ types/
  ■■■ models.d.ts
  ■■■ inertia.d.ts
```

9 Dashboard Features

Director Dashboard

- Global accreditation readiness across all programs
- Per-program progress bars (areas approved / total areas)
- Documents pending final approval
- Recent activity feed
- Export readiness PDF report

Dean Dashboard

- College-level accreditation progress
- Documents forwarded by Program Coordinators
- Area completion status per program
- Return rate and revision history stats

Program Area Coordinator Dashboard

- Progress per assigned area
- Pending submissions from Area Coordinators
- Documents returned waiting re-submission
- Upcoming deadlines per area

Area Coordinator Dashboard

- Status of all submitted documents
- Reviewer comments and return reasons
- Version history per document
- Quick upload button for pending areas

10 Example Workflow Scenario

BSIT Area 3 (Curriculum) — from initial upload to final approval.

1	Upload	Area Coordinator uploads curriculum_matrix_v1.pdf + course_syllabi.docx to BSIT Area 3. Document saved as Draft. document_versions row created with version_number = 1.
2	Submit	Area Coordinator clicks Submit. Status changes to Pending Review. WorkflowService::submit() fires DocumentSubmitted event. Program Area Coordinator notified via in-app notification and email.
3	Return	Program Area Coordinator returns with comment: "Please include the CMO alignment matrix." Status changes to Returned. workflow_actions row inserted. Area Coordinator notified.
4	Revise	Area Coordinator uploads revised file. New document_versions row with version_number = 2. Document re-submitted, status returns to Pending Review.
5	Forward to Dean	Program Area Coordinator approves and forwards. Dean receives notification with full workflow timeline visible.
6	Forward to Director	Dean adds a summary note and forwards to QUAMC Director.
7	Final Approval	QUAMC Director grants final approval. Status changes to Approved. BSIT Area 3 progress bar updates to 100% on all dashboards. All participants notified.

11 Advanced Features

Accreditation Cycle Management

Add an accreditation_cycles table with start/end dates. Documents are scoped per cycle so historical runs are preserved and comparable year-over-year.

Deadline & SLA Enforcement

Add deadline_at to areas. A Laravel scheduled job escalates overdue documents and auto-notifies the Dean and Director when submission thresholds are missed.

Document Checklist per Area

An area_checklists table allows admins to define required evidence types per area. Progress is based on checklist completion, not just document count.

Real-time Notifications (Reverb)

Use Laravel Reverb WebSockets + useEcho on the React side so workflow actions instantly update dashboards and notification bells without a page reload.

PDF Report Generation

A queued job using Laravel Snappy or DomPDF generates per-program accreditation readiness reports. Directors can export professional PDFs from the dashboard.

Virus Scanning on Upload

Integrate ClamAV via a queued job that scans uploaded files before making them accessible. Files are quarantined automatically and uploaders notified on detection.

12 Technology Stack Summary

Category	Technology	License	Cost
Frontend Framework	React 18 + TypeScript	MIT	Free
SPA Bridge	Inertia.js	MIT	Free
CSS Framework	Tailwind CSS	MIT	Free
UI Components	ShadCN/UI	MIT	Free
Build Tool	Vite	MIT	Free
Route binding	Ziggy	MIT	Free
Backend Framework	Laravel 11	MIT	Free
Language	PHP 8.3	PHP License	Free
Primary Database	MySQL 8	GPL / Commercial	Free (community)
Cache & Queues	Redis	BSD	Free (self-hosted)
File Storage (dev)	Local Disk — storage/app/	N/A	Free
File Storage (prod)	Cloudflare R2	Commercial	Free up to 10 GB
WebSockets	Laravel Reverb	MIT	Free
Queue Monitor	Laravel Horizon	MIT	Free
Search (optional)	Meilisearch	MIT	Free (self-hosted)
Web Server	Nginx	BSD	Free
Server (VPS)	DigitalOcean / Vultr / Own server	N/A	~\$10-20/mo or free
Email (notifications)	Resend / Mailgun	Commercial	Free tier (3k/mo)

All core software is MIT or similarly open-source licensed and free to self-host. File storage uses local disk during development — swap to Cloudflare R2 for production by changing one line in .env. The only mandatory recurring cost is a VPS server if the university does not have existing infrastructure. Total software licensing cost: PHP 0.